



Mid-Term Report

Summer of Science 2026

Large Language Models

Jāgu Manish
25M0612

Table of Contents

Chapter No.	Title	Page No.
1	Introduction	5
2	Python Programming Fundamentals	6
2.1	Python Basics	6
2.2	Functions	6
2.3	Loops	7
2.4	Dictionaries	7
2.5	File Handling	8
2.6	Scientific Libraries	8
2.6.1	NumPy	9
2.6.2	Pandas	9
2.6.3	Matplotlib	9
3	Artificial Intelligence and Language AI	11
3.1	Artificial Intelligence	11
3.2	Machine Learning	11
3.3	Deep Learning	12
3.4	Natural Language Processing	12
3.5	Language AI	13
4	Large Language Models	14
4.1	Introduction to Large Language Models	14
4.2	History and Evolution of Language Models	14
4.2.1	Rule-Based Systems	14
4.2.2	Statistical Language Models	15
4.2.3	Word Embeddings	15



Chapter No.	Title	Page No.
4.2.4	Recurrent Neural Networks	16
4.2.5	Long Short-Term Memory Networks (LSTMs)	16
4.2.6	Attention Mechanism	17
4.2.7	Transformer Architecture	17
4.2.8	Modern Large Language Models	18
5	Fundamental Concepts Behind Large Language Models	19
5.1	Tokens	19
5.2	Tokenization	19
5.3	Vocabulary	20
5.4	Embeddings	21
5.5	Positional Encoding	21
5.6	Attention Mechanism	22
5.7	Self-Attention	23
5.8	Multi-Head Attention	23
6	Transformer Architecture	24
6.1	Encoder Architecture	24
6.2	Decoder Architecture	25
6.3	Encoder–Decoder Models	26
6.4	BERT	26
6.5	GPT	27
6.6	Comparison between BERT and GPT	28
7	Text Generation in Large Language Models	29
7.1	Next Token Prediction	29
7.2	Context Window	30

Chapter No.	Title	Page No.
7.3	Temperature Parameter	30
7.4	Sampling Techniques	31
7.5	Beam Search	32
8	Prompt Engineering	34
8.1	Introduction to Prompt Engineering	34
8.2	Zero-Shot Prompting	34
8.3	One-Shot Prompting	34
8.4	Few-Shot Prompting	35
8.5	Chain-of-Thought Prompting	35
8.6	Prompt Design Principles and Best Practices	35
9	Challenges and Ethical Considerations	36
9.1	Hallucinations in Large Language Models	36
9.2	Bias and Fairness	36
9.3	Privacy and Security Concerns	36
9.4	Misinformation and Misuse	37
9.5	Responsible and Ethical AI Practices	37
10	APIs and LLM Integration	38
10.1	Introduction to APIs	38
10.2	REST APIs	38
10.3	JSON Data Handling	39
10.4	Making API Requests using Python	39
10.5	OpenAI APIs	39
10.6	Hugging Face APIs	40
11	Introduction to AI Applications	41

Chapter No.	Title	Page No.
11.1	Chatbots and Conversational AI	41
11.2	Streamlit Framework	41
11.3	Gradio Framework	42
11.4	Building Simple AI Applications	43
12	Mid-Term Progress Summary	44
13	Conclusion	45
14	References	46

1 Introduction

Artificial Intelligence has changed the way people relate to computers and digital tools. Its core goal is to create systems that can think and learn—systems that reason, solve problems, make decisions, and understand language the way humans do (Russell & Norvig, 2021). Over just the last decade, huge jumps in computing power, the explosion of accessible data, and sharper machine learning methods have pushed AI forward at breakneck speed.

Within this field, Natural Language Processing (NLP) stands out. NLP aims to help computers really grasp and use human language. Early attempts relied on tedious handcrafted rules and statistical approaches, mostly for tasks like translation or sorting text. While those efforts sometimes worked well in narrow cases, they couldn't keep up when language turned messy or unpredictable (Jurafsky & Martin, 2023). Then deep learning hit the scene. Neural nets like RNNs and LSTMs help manage language as sequences, which is essential for understanding context. Still, these models hit a wall handling longer text and often lagged in training (Goodfellow et al., 2016). The real turning point came with the introduction of transformers by Vaswani et al. (2017). Transformers ditched strict sequential processing. With self-attention, they look at words in relation to every other word all at once, grabbing meaning even across huge gaps in text. This breakthrough led to faster, more powerful language models. Transformers paved the way for Large Language Models (LLMs). These models soak up knowledge from vast troves of books, articles, webpages, and more. By crunching the patterns between words and ideas, LLMs handle a staggering range of tasks: answering questions, writing summaries, generating code, chatting in natural language, translating text—you name it (Brown et al., 2020). And notably, they do all this with little or no extra training for each new job.

Recent years have seen several big names shape this field. BERT, for instance, introduced a two-way approach to reading text that significantly improved language understanding (Devlin et al., 2019). The GPT series took off by focusing even more on generating smooth, human-like text and excelled at conversational tasks by pretraining on massive datasets (Radford et al., 2019; Brown et al., 2020). These breakthroughs haven't just influenced research; they've entered real-life work in businesses, healthcare, education, and science. Still, LLMs aren't perfect. They sometimes produce false or “hallucinated” answers, repeat old biases in their data, and raise privacy and energy-use concerns. Researchers are grappling with these issues, working to make models more trustworthy, understandable, and responsibly deployed (Bender et al., 2021).

This report is a conceptual survey of Large Language Models and the surrounding technologies, based on work as part of Summer of Science 2026 LLM project. It covers the basics of Python programming, core machine learning concepts, the rise of language models, how transformers work, prompt engineering, ethics, and how to actually use LLMs via APIs. The aim is to build a solid theoretical base—one that lays the groundwork for future hands-on AI projects.

2 Python Programming Fundamentals

Python stands out as a high-level, interpreted programming language that's found everywhere—software development, data science, artificial intelligence, and scientific computing. The language is famous for its clear, straightforward syntax, making code easy to read and write. With a large library of built-in modules and third-party packages, Python provides developers with plenty of tools to tackle nearly any project. It runs on just about any platform, and its active community keeps it well-supported and up to date. Together, these features explain why both academia and industry rely on Python so heavily (Van Rossum and Drake, 2009).

2.1 Python Basics

Python programs consist of statements executed sequentially by an interpreter. The language supports multiple programming paradigms, including procedural, object-oriented, and functional programming. Variables in Python are dynamically typed, allowing programmers to assign values without explicitly declaring data types.

Python provides built-in data types such as integers, floating-point numbers, strings, lists, tuples, dictionaries, and sets.

Example:

```
name = "Summer of Science"
year = 2026
topic = "Large Language Models"

print(name)
print(year)
print(topic)

Summer of Science
2026
Large Language Models
```

The above example demonstrates variable declaration and output generation using the `print()` function.

2.2 Functions

Functions are reusable blocks of code designed to perform specific tasks. They help improve code modularity, readability, and maintainability. Python functions are defined using the `def` keyword and may accept parameters and return processed values.

Example:

```
def square(number):  
    return number * number  
result = square(5)  
print(result)
```

25

In this example, the function `square()` accepts an integer as input and returns its square.

2.3 Loops

Loops are control structures used to execute a set of statements repeatedly until a specified condition is met. Python supports two primary looping constructs:

1. for loop
2. while loop

The for loop is commonly used when the number of iterations is known in advance, whereas the while loop continues until a condition evaluates false.

Example:

```
for i in range(5):  
    print(i)
```

0
1
2
3
4

The `range(5)` function generates a sequence of integers from 0 to 4, which are iteratively printed.

2.4 Dictionaries

Dictionaries are mutable data structures that store information as key-value pairs. They enable efficient retrieval, insertion, and modification of data using unique keys.

Dictionaries are widely used in machine learning workflows to store model parameters, configuration settings, and processed datasets.

Example:

```
course = {  
    "Name": "Summer of Science",  
    "Year": 2026,  
    "Topic": "Large Language Models"  
}  
  
print(course["Name"])  
  
Summer of Science
```

In this example, the value associated with the key "Name" is retrieved from the dictionary.

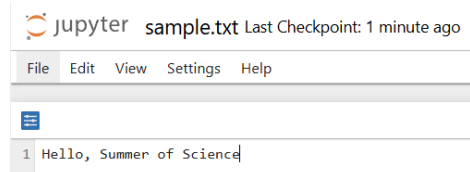
2.5 File Handling

File handling enables programs to create, read, modify, and store information in external files. Python supports various file operations through built-in functions such as `open()`, `read()`, `write()`, and `close()`.

Proper file handling is essential in data science and machine learning applications, where datasets are frequently accessed from text files, CSV files, and databases.

Example:

```
file = open("sample.txt", "w")  
file.write("Hello, Summer of Science")  
file.close()
```



The code above creates a text file named `sample.txt` and writes a message to it.

2.6 Scientific Libraries

Python provides numerous scientific computing libraries that simplify data analysis, numerical computation, and visualization tasks. Three widely used libraries are NumPy, Pandas, and Matplotlib.

2.6.1 NumPy

NumPy (Numerical Python) is a fundamental library for scientific computing in Python. It provides support for multidimensional arrays, mathematical functions, linear algebra operations, random number generation, and statistical computations.

NumPy arrays offer significant performance advantages over traditional Python lists due to optimized memory management and vectorized operations.

Example:

```
import numpy as np

arr = np.array([1, 2, 3, 4])
print(arr * 2)

[2 4 6 8]
```

The example demonstrates element-wise multiplication using a NumPy array.

2.6.2 Pandas

Pandas is an open-source library designed for data manipulation and analysis. It introduces two important data structures:

- Series
- DataFrame

Pandas enables efficient handling of structured datasets and provides functionalities for filtering, grouping, cleaning, and transforming data.

Example:

```
import pandas as pd

data = {
    'Name': ['A', 'B'],
    'Marks': [85, 92]
}

df = pd.DataFrame(data)
print(df)
```

	Name	Marks
0	A	85
1	B	92

This example creates a simple DataFrame containing student names and marks.

2.6.3 Matplotlib

Matplotlib is a comprehensive visualization library for generating static, animated, and interactive plots in Python. It is extensively employed for exploratory data analysis, model evaluation, and scientific visualization.

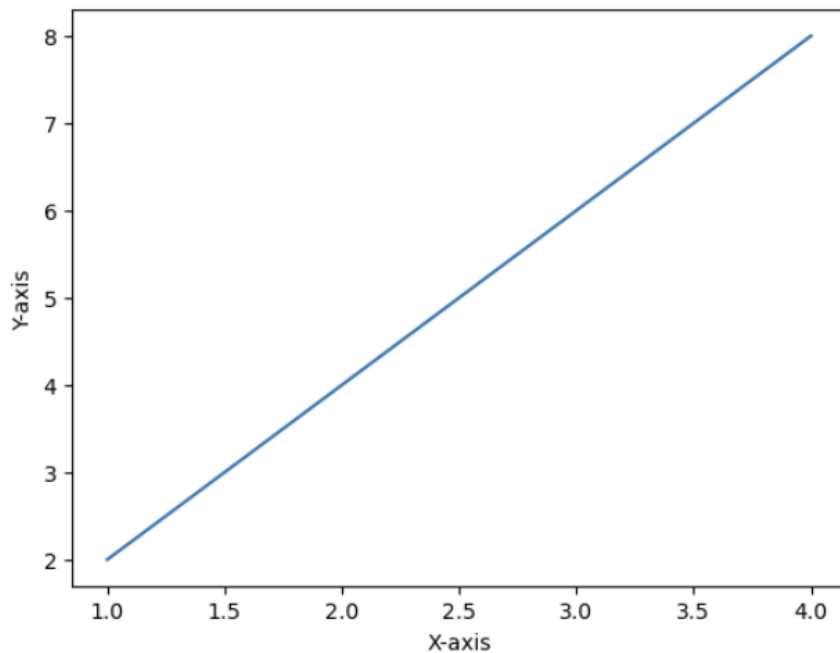
Common plot types include line plots, bar charts, histograms, scatter plots, and pie charts.

Example:

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4]
y = [2, 4, 6, 8]

plt.plot(x, y)
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.show()
```



The above example generates a simple line graph representing the relationship between two variables.

3 Artificial Intelligence and Language AI

Artificial Intelligence (AI) has become one of the most influential technological advancements of the twenty-first century. It encompasses a broad range of computational techniques that enable machines to perform tasks traditionally associated with human intelligence. Over the years, the field has evolved from simple rule-based systems to sophisticated deep learning architectures capable of understanding and generating human language. This section presents an overview of Artificial Intelligence, Machine Learning, Deep Learning, Natural Language Processing, and Language AI, which collectively form the foundation of modern Large Language Models.

3.1 Artificial Intelligence

Artificial Intelligence (AI) refers to the capability of computer systems to perform tasks that typically require human intelligence, such as learning, reasoning, problem-solving, perception, and decision-making. The primary objective of AI is to develop intelligent agents that can analyze their environment and take actions to maximize the chances of achieving specific goals (Russell and Norvig, 2021).

The concept of Artificial Intelligence was formally introduced by John McCarthy during the Dartmouth Conference in 1956, where AI was envisioned as a discipline aimed at understanding and replicating human intelligence using machines (McCarthy et al., 1955). Since then, AI has expanded into numerous domains, including healthcare, finance, robotics, autonomous systems, and natural language understanding.

AI systems can generally be categorized into two broad classes:

- **Narrow AI (Weak AI):** Designed to perform specific tasks, such as image recognition, recommendation systems, and voice assistants.
- **General AI (Strong AI):** A hypothetical form of intelligence capable of understanding, learning, and performing any intellectual task that a human can accomplish.

Current AI applications primarily fall into the category of Narrow AI and have demonstrated remarkable success across various real-world scenarios.

3.2 Machine Learning

Machine Learning (ML) is a subfield of Artificial Intelligence that focuses on developing algorithms capable of learning patterns from data and making predictions or decisions without being explicitly programmed for every task (Mitchell, 1997).

Unlike traditional rule-based systems, machine learning algorithms improve their performance by analyzing historical data and identifying underlying relationships within the dataset. Machine learning techniques have become increasingly important due to the



growing availability of large datasets and advancements in computational power.

Machine learning methods are generally classified into three categories:

3.2.1 Supervised Learning: In supervised learning, models are trained on labeled datasets, where both the input features and the expected outputs are available. Examples include classification and regression problems. Examples include spam email detection, House price prediction, and disease diagnosis.

3.2.2 Unsupervised Learning: Unsupervised learning involves discovering hidden patterns or structures from unlabeled data. Examples include Customer segmentation, Clustering analysis, and Dimensionality reduction

3.2.3 Reinforcement Learning: Reinforcement learning enables an agent to learn optimal actions through interactions with an environment by maximizing cumulative rewards. Examples include Autonomous vehicles, Robotics, and game-playing agents

Machine learning serves as an essential foundation for many modern AI applications, including speech recognition systems, recommendation engines, and predictive analytics.

3.3 Deep Learning

Deep Learning (DL) is a specialized branch of machine learning that employs artificial neural networks containing multiple hidden layers to model complex patterns in data (Goodfellow et al., 2016). Deep learning algorithms are inspired by the structure and functioning of the human brain, where interconnected neurons process information and transmit signals. By increasing the number of layers within neural networks, deep learning models can automatically extract hierarchical representations from raw input data. Several neural network architectures have been developed for different applications, including Feedforward Neural Networks, Convolutional Neural Networks (CNNs), Recurrent Neural Networks (RNNs), Long Short-Term Memory Networks (LSTMs), and Transformer Networks.

Deep learning has significantly improved the performance of tasks such as image classification, speech recognition, object detection, and natural language processing. The emergence of transformer architectures has further revolutionized language-related applications and facilitated the development of Large Language Models.

3.4 Natural Language Processing

Natural Language Processing (NLP) is a branch of Artificial Intelligence concerned with enabling computers to understand, interpret, process, and generate human languages in a meaningful manner (Jurafsky and Martin, 2023).

Human languages are inherently complex due to ambiguity, variations in syntax, semantics, context, and cultural influences. NLP techniques aim to bridge the communication gap between humans and machines by allowing systems to analyze textual and spoken information. Common NLP tasks include Text classification, Machine translation, Named Entity Recognition, Sentiment Analysis, Speech Recognition, Question Answering, Text Summarization.

Earlier NLP systems heavily relied on handcrafted linguistic rules and statistical approaches. However, these methods often struggled to capture long-range dependencies and contextual meanings. The integration of deep learning techniques, particularly transformer-based architectures, has substantially enhanced the performance of NLP systems and enabled more accurate language understanding capabilities.

3.5 Language AI

Language AI is an advanced area within Artificial Intelligence that focuses on developing systems capable of understanding, generating, and interacting through natural human language. It combines concepts from Natural Language Processing, deep learning, and large-scale language modeling to facilitate intelligent communication between humans and machines. Language AI systems have become increasingly sophisticated due to the availability of massive datasets and powerful computational infrastructures. Modern Language AI applications include Conversational Chatbots, Virtual Assistants, Automatic Content Generation, Code Generation Systems, Intelligent Search Engines, Document Summarization Tools, and Language Translation Platforms.

Large Language Models such as GPT, BERT, LLaMA, and Gemini represent significant milestones in the advancement of Language AI. These models are trained on extensive corpora containing billions of words and can perform numerous language-related tasks with minimal task-specific fine-tuning.

Despite their remarkable capabilities, Language AI systems continue to face challenges related to hallucinations, fairness, privacy, explainability, and responsible deployment. Ongoing research aims to address these limitations while improving the reliability and safety of language-based intelligent systems.

4 Large Language Models

4.1 Introduction to Large Language Models

Large Language Models (LLMs) are advanced artificial intelligence systems designed to understand, generate, and manipulate human language. They are typically built using deep neural network architectures and trained on massive collections of textual data obtained from books, articles, websites, scientific literature, and other publicly available resources. Through this extensive training process, LLMs learn statistical relationships between words, phrases, and contextual patterns, enabling them to perform a wide variety of language-related tasks (Brown et al., 2020).

Unlike traditional Natural Language Processing models that are developed for specific tasks, modern LLMs possess the capability to generalize across multiple domains with little or no task-specific training. They can answer questions, summarize lengthy documents, generate source code, translate languages, write essays, assist in research activities, and engage in conversational interactions. These capabilities have contributed significantly to the widespread adoption of LLMs in education, healthcare, software engineering, customer support, and scientific research.

The development of LLMs represents a culmination of decades of advancements in language modeling, neural networks, and deep learning methodologies. The evolution of language models from simple rule-based approaches to transformer-based architectures has been instrumental in achieving the remarkable capabilities observed in current-generation language models.

4.2 History and Evolution of Language Models

The field of language modeling has undergone substantial changes over the past several decades. Researchers have continuously sought methods to enable machines to process human language more effectively. The evolution of language models can be broadly divided into several stages.

4.2.1 Rule-Based Systems

Rule-based systems are among the earliest approaches to Natural Language Processing. These systems relied heavily on manually designed linguistic rules, dictionaries, grammar structures, and expert knowledge to analyze and generate text.

In rule-based approaches, linguists and domain experts explicitly encoded syntactic and semantic rules into computer programs. For example, translation systems developed during the early years of NLP utilized predefined grammar rules to convert text from one language to another.

Although rule-based methods were useful for well-defined and constrained tasks, they exhibited several limitations. Developing comprehensive linguistic rules for multiple languages was labor-intensive, expensive, and difficult to scale. Additionally, these systems struggled to handle ambiguity, slang expressions, contextual meanings, and previously unseen sentence structures (Jurafsky and Martin, 2023).

The limitations associated with handcrafted rules motivated researchers to investigate data-driven approaches capable of learning linguistic patterns automatically from textual corpora.

4.2.2 Statistical Language Models

Statistical Language Models (SLMs) emerged as an alternative to rule-based systems by utilizing probability theory to estimate the likelihood of word sequences.

The primary objective of a statistical language model is to predict the probability of observing a sequence of words within a language. These models assume that words appearing together frequently in large text collections are likely to occur together in future sentences.

One commonly employed approach was the n-gram model, which predicts the next word based on the preceding $n-1$ words. For instance, a trigram model estimates the probability of a word using the previous two words.

Although statistical models improved language processing capabilities, they suffered from several shortcomings. They required extensive memory to store probability distributions and were unable to effectively capture long-range dependencies between distant words in a sentence. Consequently, researchers explored neural network-based approaches for language modeling.

4.2.3 Word Embeddings

Traditional statistical approaches treated words as discrete symbols, ignoring their semantic relationships. To overcome this limitation, researchers introduced distributed word representations known as word embeddings. Word embeddings map words into continuous vector spaces, where semantically related words occupy nearby positions. These representations enable machine learning algorithms to understand similarities and relationships among words. One of the earliest influential embedding techniques was Word2Vec, proposed by Mikolov et al. (2013). Word2Vec introduced two architectures:

1. Continuous Bag of Words (CBOW)
2. Skip-Gram

Another popular approach was GloVe (Global Vectors), developed by Pennington et al. (2014), which utilized global word co-occurrence statistics to learn meaningful word representations.

Word embeddings significantly enhanced language models' ability to capture semantic information and laid the foundation for neural language processing techniques.

4.2.4 Recurrent Neural Networks

Recurrent Neural Networks (RNNs) were introduced to address the shortcomings of traditional statistical models by incorporating sequential information into neural architectures. Unlike feedforward neural networks, RNNs maintain hidden states that preserve information from previous time steps. This property enables RNNs to process sequential data such as text, speech, and time-series observations. RNNs demonstrated promising performance across tasks such as language modeling, Machine translation, Speech recognition, and sentiment analysis.

However, standard RNNs encountered difficulties learning long-term dependencies due to vanishing and exploding gradients during training (Goodfellow et al., 2016).

4.2.5 Long Short-Term Memory Networks (LSTMs)

Long Short-Term Memory Networks (LSTMs) were proposed by Hochreiter and Schmidhuber (1997) to overcome the limitations of conventional RNNs.

LSTMs introduce specialized memory cells and gating mechanisms that regulate the flow of information through the network. These components include:

1. Input Gate
2. Forget Gate
3. Output Gate

The architecture enables the network to retain important contextual information over extended sequences while discarding irrelevant details.

LSTMs achieved state-of-the-art results in several NLP applications, including speech synthesis, text generation, handwriting recognition, and machine translation. Despite these advancements, their sequential processing nature resulted in increased computational costs when dealing with extremely large datasets.

4.2.6 Attention Mechanism

The attention mechanism represented a major breakthrough in sequence modeling by enabling neural networks to selectively focus on relevant portions of the input sequence. Instead of compressing all contextual information into a single fixed-length vector, attention mechanisms assign varying importance scores to different words within a sentence. For example, while translating a sentence from English to French, the model can dynamically attend to the words most relevant to the current translation step.

Attention significantly improved translation quality and contextual understanding while reducing the information bottleneck encountered in recurrent architectures. The success of attention mechanisms ultimately paved the way for transformer-based language models.

4.2.7 Transformer Architecture

A significant milestone in Natural Language Processing was achieved with the introduction of the Transformer architecture by Vaswani et al. (2017). The transformer model departed from traditional recurrent architectures and relied entirely on self-attention mechanisms.

The architecture consists of two principal components:

1. Encoder
2. Decoder

Transformers process input sequences in parallel, making them substantially more efficient than recurrent networks for training on large datasets.

Key advantages of transformers include:

- Parallel computation
- Better handling of long-range dependencies
- Improved scalability
- Enhanced contextual understanding

The transformer architecture has become the foundation for nearly all modern Large Language Models.

4.2.8 Modern Large Language Models

Modern Large Language Models are primarily based on transformer architectures and trained using self-supervised learning objectives on billions or even trillions of textual tokens.

Several influential models have emerged over the past few years.

- BERT (Devlin et al., 2019) introduced bidirectional contextual representations and demonstrated exceptional performance on language understanding tasks.
- GPT-3 (Brown et al., 2020) showcased the effectiveness of scaling model parameters and training datasets, enabling impressive zero-shot and few-shot learning capabilities.

Subsequent developments include GPT-4, PaLM, LLaMA, Gemini, Claude, and Mistral models, which continue to push the boundaries of language understanding and generation.

Modern LLMs have become versatile tools that support scientific research, software development, education, healthcare, and numerous other domains. However, challenges related to computational costs, hallucinations, bias, and ethical concerns remain active areas of investigation.

5 Fundamental Concepts Behind LLMs

Large Language Models (LLMs) are built upon several fundamental concepts that enable them to understand, represent, and generate human language effectively. These concepts provide the foundation for modern transformer-based architectures and play a crucial role in determining the performance of language models. This section discusses the essential building blocks underlying LLMs, including tokens, tokenization, embeddings, positional encoding, and attention mechanisms.

5.1 Tokens

Tokens are the smallest units of text processed by a language model during training and inference. Unlike humans, who naturally interpret complete words or sentences, language models analyze textual data by dividing it into smaller components called tokens.

A token may represent a complete word, a subword segment, a single character, punctuation marks, numerical values, or special symbols. For example, the sentence "Large Language Models are powerful." may be tokenized as:

["Large", "Language", "Models", "are", "powerful", "."]

Alternatively, subword tokenization techniques may split a word into smaller meaningful parts. Example: Unbelievable as ["Un", "believ", "able"]

The number of tokens a language model processes determines its computational requirements and context limitations.

5.2 Tokenization

Tokenization is the process of converting raw textual input into a sequence of tokens that can be understood and processed by machine learning models. Tokenization serves as an important preprocessing step in Natural Language Processing pipelines. Several tokenization approaches have been proposed.

1. **Word-Level Tokenization:** Entire words are treated as individual tokens.

Example:

Input: I love Artificial Intelligence
Tokens: ["I", "love", "Artificial", "Intelligence"]

Advantages include being easy to understand. Disadvantages include large vocabulary size and difficulty handling unseen words.

2. **Character-Level Tokenization:** Each character becomes a token.

Example:

Input: "Science"
Tokens: ['S','c','i','e','n','c','e']

Advantages include a small vocabulary; disadvantages include longer input sequences.

3. **Subword Tokenization:** Modern LLMs primarily use subword tokenization techniques.

Popular algorithms include Byte Pair Encoding (BPE), SentencePiece, WordPiece.

Example:

Input: unhappiness
Tokens: ["un", "happy", "ness"]

Subword methods efficiently balance vocabulary size and semantic understanding.

5.3 Vocabulary

Vocabulary refers to the complete collection of unique tokens that a language model recognizes during training and inference.

Each token within the vocabulary is assigned a unique numerical identifier.

Example:

Token	ID
The	100
AI	241
learns	512
fast	790

Modern language models often employ very large vocabularies.

Example:

Model	Vocabulary Size
BERT	~30,000
GPT-2	~50,000
GPT-3	~50,000+

The vocabulary size influences model efficiency, memory requirements, and tokenization performance.

5.4 Embeddings

Computers cannot directly interpret textual data. Therefore, tokens must be converted into numerical representations before they can be processed by neural networks. Embeddings are dense numerical vectors that capture semantic and syntactic information about words. Unlike one-hot encoding, embeddings place semantically related words close together in vector space. For example:

Word	Embedding Vector
King	[0.25,0.71,0.34,...]
Queen	[0.29,0.74,0.36,...]
Prince	[0.22,0.69,0.30,...]

An interesting property of embeddings is their ability to preserve relationships.

Example: King - Man + Woman $\approx$ Queen

Popular embedding approaches include Word2Vec, GloVe, FastText.

Modern transformer models learn contextual embeddings dynamically during training.

5.5 Positional Encoding

Transformers process all input tokens simultaneously rather than sequentially. As a result, transformers do not inherently possess information regarding the order of words within a sentence. To address this limitation, positional encoding is introduced. Positional encoding provides information about each token's position within the sequence. Consider the sentences

“Cats chase mice” and “Mice chase cats.” Although both sentences contain identical words, their meanings differ because of word order. Positional encoding allows the transformer to distinguish between these sequences. The original Transformer model proposed sinusoidal positional encoding.

The equations are:

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d}}\right)$$

$$PE(pos, 2i + 1) = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

where,

- pos = position index
- i = embedding dimension
- d = embedding size

Positional encoding preserves sequence information while maintaining parallel processing capabilities.

5.6 Attention Mechanism

The attention mechanism is one of the most significant innovations in deep learning for Natural Language Processing. Attention enables a model to selectively focus on important words when generating predictions. For example, consider the sentence, "The cat sat on the mat because **it** was comfortable."

To understand the word **it**, the model must determine that it refers to **the mat**. Attention computes relevance scores among tokens and assigns higher importance to contextually related words. The attention score is calculated as:

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

where:

- Q = Query
- K = Key
- V = Value

Attention improves contextual understanding and long-range dependency modeling.

5.7 Self-Attention

Self-attention is a specialized form of attention used extensively in transformer architectures. In self-attention, each token attends to all other tokens within the same sequence. Consider, "The animal didn't cross the road because **it** was tired."

To correctly interpret **it**, the model evaluates relationships among all words in the sentence.

The self-attention process involves:

Step 1: Generate Queries

Step 2: Generate Keys

Step 3: Generate Values

Step 4: Calculate similarity scores

Step 5: Normalize scores using Softmax

Step 6: Compute weighted representations

Self-attention allows transformers to capture both local and global contextual relationships efficiently.

5.8 Multi-Head Attention

Instead of learning a single attention representation, transformers employ multiple attention heads simultaneously. Each head learns different linguistic characteristics. For example:

Head 1: Subject relationships

Head 2: Verb dependencies

Head 3: Semantic meaning

Head 4: Positional relationships

The outputs from multiple heads are concatenated and projected into a final representation.

Advantages of multi-head attention include improved contextual understanding, better representation learning, enhanced ability to capture complex language patterns, and increased model robustness.

Multi-head attention constitutes a fundamental component of modern transformer architectures and significantly contributes to the exceptional performance of contemporary Large Language Models.

6 Transformer Architecture

The Transformer architecture has revolutionized Natural Language Processing (NLP) by overcoming many of the limitations of recurrent neural networks and Long Short-Term Memory (LSTM) networks. Introduced by Vaswani et al. (2017), the transformer relies entirely on self-attention mechanisms, enabling parallel computation and efficient learning of long-range dependencies within textual sequences. Unlike traditional sequence models that process information sequentially, transformers process all input tokens simultaneously, significantly reducing training time while improving contextual understanding. Due to these advantages, transformer architectures have become the foundation of most modern Large Language Models.

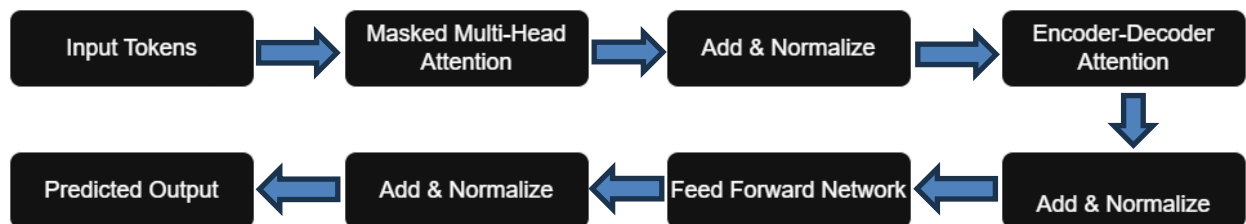
6.1 Encoder Architecture

The encoder processes the input sequence and generates contextualized representations for each token. It consists of multiple identical layers stacked sequentially, with each layer containing two primary sub-components:

- Multi-Head Self-Attention Layer
- Feed Forward Neural Network

Initially, input tokens are converted to embeddings, and positional encodings are added to preserve sequence-order information. The embeddings are then passed through the self-attention mechanism, allowing each token to attend to all other tokens within the sequence. The output of the attention layer is passed through residual connections and layer normalization to improve training stability. Subsequently, a feed-forward neural network is applied independently to each token representation.

The operations within an encoder layer can be summarized as follows:



The decoder architecture is commonly employed in text generation tasks, including machine translation, summarization, and conversational AI systems.

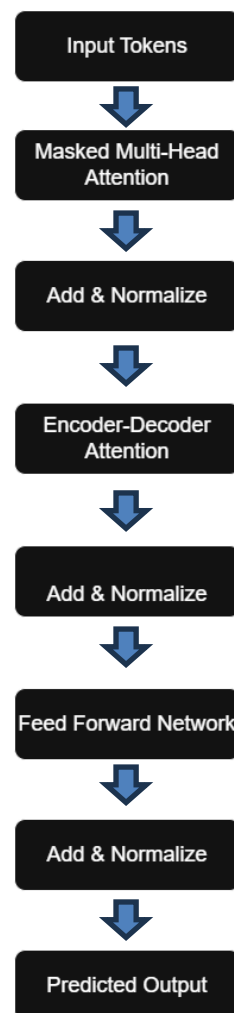
6.2 Decoder Architecture

The decoder component generates output sequences based on information received from the encoder and previously generated tokens. Similar to the encoder, the decoder consists of multiple stacked layers. Each decoder layer contains three major components:

- Masked Multi-Head Self-Attention
- Encoder–Decoder Attention
- Feed Forward Neural Network

Masked self-attention ensures that predictions are generated autoregressively by preventing access to future tokens during training. For example, when predicting the fourth word in a sentence, the decoder can only utilize information from the first three words. The encoder-decoder attention mechanism allows the decoder to focus on relevant portions of the encoded input sequence.

The processing pipeline of the decoder can be represented as:

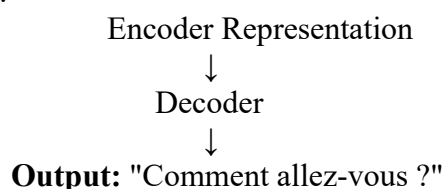


6.3 Encoder–Decoder Models

Encoder-decoder models combine the strengths of both encoder and decoder architectures. The encoder transforms the input sequence into contextual representations, while the decoder generates the desired output sequence token by token. These architectures have demonstrated remarkable success in sequence-to-sequence learning problems. Common applications include Machine Translation, Text Summarization, Speech Recognition, Question Answering, and Dialogue Systems. One well-known example is the Transformer model proposed by Vaswani et al. (2017), which employs both an encoder and a decoder stack.

Example:

Input: "How are you?"



Encoder-decoder models excel at tasks that require transforming an input sequence into another sequence. However, some language tasks require only understanding or only text-generation capabilities. This observation led researchers to develop specialized architectures such as BERT and GPT.

6.4 BERT

BERT (Bidirectional Encoder Representations from Transformers) was introduced by Devlin et al. (2019) and is based entirely on the encoder component of the transformer architecture. The primary innovation of BERT lies in its bidirectional training strategy. Traditional language models process text either from left-to-right or right-to-left. In contrast, BERT simultaneously considers both preceding and succeeding words while learning contextual representations. For example: Sentence: "The bank is located near the river."

BERT analyzes the complete sentence before assigning meaning to the word bank, enabling it to distinguish between a financial institution and the side of a river. BERT is pretrained using two objectives:

1. **Masked Language Modeling (MLM):** Random words are hidden during training.

Example:

Input: The cat sat on the [MASK].
Prediction: mat

2. **Next Sentence Prediction (NSP):** The model determines whether two sentences logically follow one another.

Example:

Sentence A: I am studying Artificial Intelligence.

Sentence B: I want to build language models.

Output: True

BERT is widely used for language understanding tasks such as Text Classification, Sentiment Analysis, Named Entity Recognition, Question Answering, and Information Retrieval.

Several variants of BERT have been proposed, including RoBERTa, DistilBERT, ALBERT, and ELECTRA.

6.5 GPT

GPT (Generative Pre-trained Transformer) was introduced by OpenAI and is primarily based on the transformer architecture's decoder component. Unlike BERT, GPT processes text in an autoregressive manner. It predicts the next token using previously observed tokens.

Example:

Input: Artificial Intelligence is changing

Prediction: the

Next prediction: world

GPT models are pretrained on extensive text corpora using self-supervised learning. The training objective can be represented as:

Maximize:

$$P(w_1, w_2, \dots, w_n)$$

which is decomposed into:

$$P(w_1)$$

$$\times P(w_2|w_1)$$

$$\times P(w_3|w_1, w_2)$$

$$\times \dots$$

This approach enables GPT models to generate coherent and contextually relevant text.

Notable GPT models include GPT-1 (2018), GPT-2 (2019), GPT-3 (2020), GPT-4 (2023), GPT-4o, GPT-5 family

GPT models have demonstrated strong performance in Text Generation, Code Generation, Summarization, Conversational Agents, Content Creation, and Research Assistance. The scalability of GPT models has played a major role in advancing modern Large Language Models.

6.6 Comparison between BERT and GPT

Although both BERT and GPT are transformer-based architectures, they are designed for different objectives.

Feature	BERT	GPT
Architecture	Encoder Only	Decoder Only
Processing Direction	Bidirectional	Left-to-Right
Primary Objective	Understanding Text	Generating Text
Training Method	Masked Language Modeling	Next Token Prediction
Best Applications	Classification, QA	Chatbots, Content Generation
Context Awareness	Full Sentence Context	Previous Tokens Only
Examples	BERT, RoBERTa	GPT-3, GPT-4

Both architectures have significantly influenced the development of modern language models. BERT excels at extracting contextual information for understanding tasks, whereas GPT excels at generating coherent, human-like text. Recent advancements in Large Language Models continue to build upon these transformer-based foundations, incorporating larger datasets, increased computational resources, and improved training methodologies.

7 Text Generation in Large Language Models

Text generation is one of the most remarkable capabilities exhibited by Large Language Models (LLMs). Unlike traditional Natural Language Processing systems designed to perform specific tasks, modern language models can generate coherent, contextually relevant, and human-like text by predicting word sequences based on previously observed data. This capability has enabled the development of conversational assistants, content-generation systems, code-completion tools, machine translation models, and numerous other applications. Text generation in LLMs is primarily based on probabilistic prediction mechanisms, where the model estimates the likelihood of subsequent tokens and selects the most appropriate continuation based on predefined decoding strategies. Several important concepts influence the quality, diversity, and coherence of generated text.

7.1 Next Token Prediction

Next-token prediction is the fundamental learning objective used by many autoregressive language models, including the GPT family. The primary goal is to predict the most probable token that follows a given sequence of tokens. During training, the model is presented with a sequence of words and learns to estimate the probability distribution of the next word given contextual information. Through exposure to billions of text samples, the model gradually acquires knowledge about grammar, syntax, semantics, and linguistic patterns.

For example, consider the sentence, Artificial Intelligence is transforming the _____.

The language model may assign probabilities to several candidate tokens:

Candidate Token	Probability
world	0.68
industry	0.17
future	0.09
society	0.04
economy	0.02

Since the word "**world**" has the highest probability, it is selected as the next generated token.

After generating one token, the process repeats iteratively until an end-of-sequence token is encountered or a predefined length limit is reached. This sequential prediction mechanism enables language models to generate paragraphs, articles, computer programs, and conversational responses. The effectiveness of next-token prediction depends significantly on

the size of the training corpus, the model architecture, and the decoding strategy used during inference (Brown et al., 2020).

7.2 Context Window

The context window refers to the amount of textual information that a language model can consider simultaneously while generating a response.

Since language models do not possess persistent memory in the traditional sense, they rely on the tokens present within the context window to maintain coherence and relevance.

For example:

Prompt: The capital city of France is Paris. It is famous for the Eiffel Tower. Millions of tourists visit the city every year. Which city is being discussed?

Output: Paris

The model correctly identifies "Paris" because the relevant information is available within its context window. If important details exceed the maximum context length, earlier information may be truncated or forgotten. Different language models support varying context sizes.

Examples include:

Model	Approximate Context Window
BERT	512 Tokens
GPT-2	1024 Tokens
GPT-3	2048 Tokens
GPT-4	Up to 128,000 Tokens*
Claude 3	Up to 200,000 Tokens*

*Values may vary depending on model versions and deployment configurations.

Larger context windows enable models to process lengthy documents, research papers, source code repositories, and extended conversations more effectively.

7.3 Temperature Parameter

The temperature parameter controls the randomness and creativity of generated text. Mathematically, temperature modifies the probability distribution produced by the model before token selection. Lower temperature values make the probability distribution sharper,

leading to more deterministic, focused outputs. Higher temperature values flatten the distribution, encouraging more diverse and creative responses. Typical values include:

Temperature	Characteristics
0.0	Deterministic
0.2	Highly focused
0.7	Balanced
1.0	Diverse
>1.0	Highly creative

Example prompt: Suggest a title for an article on Artificial Intelligence.

Temperature = 0.2 || Output: Introduction to Artificial Intelligence

Temperature = 1.0 || Output: Beyond Algorithms: Exploring the Future of Intelligent Machines

Temperature is commonly adjusted depending on the intended application. Tasks such as summarization and question answering often benefit from lower temperatures, whereas creative writing applications may utilize higher values.

7.4 Sampling Techniques

Sampling techniques determine how the next token is selected from the predicted probability distribution. Different sampling methods provide varying trade-offs between diversity and coherence.

1. Greedy Sampling: Greedy sampling selects the token with the highest probability at every prediction step.

Word	Probability
cat	0.70
dog	0.20
bird	0.10

Selected token: cat

Advantages include speed and determinism. Disadvantages include repetitive outputs and reduced creativity.

2. Random Sampling: Random sampling selects tokens according to their assigned probabilities. Although this method increases diversity, it may occasionally generate incoherent text.

3. Top-k Sampling: Top-k sampling limits candidate tokens to the top **k** most probable words. **Example:** $k = 3$

Candidates: [cat, dog, bird]

The final token is randomly selected from these candidates. Advantages include improved diversity and better output quality

4. Top-p Sampling (Nucleus Sampling): Top-p sampling dynamically selects the smallest subset of words whose cumulative probability exceeds a specified threshold.

Example: $p = 0.9$

Candidate tokens:

Token	Probability
cat	0.55
dog	0.20
bird	0.10
fish	0.05

Cumulative probability: $0.55 + 0.20 + 0.10 + 0.05 = 0.90$

Selection occurs only among these tokens. Top-p sampling is widely adopted in modern conversational language models because it effectively balances coherence and creativity.

7.5 Beam Search

Beam Search is a decoding strategy that improves sequence generation quality by maintaining multiple candidate sequences simultaneously. Instead of selecting only the single most probable token at each step, beam search keeps track of several high-scoring partial sequences known as beams. Suppose the beam width is set to 3. Initial prompt is Artificial Intelligence, and the possible continuations will be,

Beam 1: is changing the world

Beam 2: has become increasingly important

Beam 3: can assist researchers

At each generation step, the algorithm expands all candidate sequences and retains only the highest-scoring beams. Advantages of Beam Search include Improved sentence fluency, Better sequence optimization, and reduced likelihood of poor local decisions. However, beam search requires greater computational resources and may occasionally produce repetitive outputs. Beam search is frequently employed in applications such as machine translation, speech recognition, and text summarization, where maintaining grammatical correctness and semantic consistency is particularly important.

8 Prompt Engineering

Prompt engineering refers to the process of designing and optimizing inputs (prompts) provided to Large Language Models (LLMs) to obtain accurate, relevant, and contextually appropriate outputs. Effective prompting techniques can significantly improve model performance without modifying the underlying model parameters. Prompt engineering has become an important aspect of utilizing modern LLMs for various applications, including question answering, text summarization, code generation, and conversational systems.

8.1 Introduction to Prompt Engineering

Prompt engineering involves carefully crafting instructions, questions, or examples that guide a language model toward producing desired responses. Well-structured prompts help reduce ambiguity and improve output quality. As LLMs continue to evolve, prompt engineering has emerged as a practical approach for leveraging their capabilities efficiently (Brown et al., 2020).

8.2 Zero-Shot Prompting

Zero-shot prompting refers to providing a task description to a language model without including any examples. The model relies solely on the knowledge acquired during pretraining to generate an appropriate response.

Example:

Prompt: Translate the following sentence into French: "Good morning."

Output: Bonjour.

Zero-shot prompting is useful when examples are unavailable or when quick task adaptation is required.

8.3 One-Shot Prompting

One-shot prompting provides the model with a single example before presenting the actual task. The example helps the model understand the expected input-output format. One-shot prompting often improves performance for structured tasks. Example includes,

Prompt: Positive → Happy; Negative → Sad; Neutral → ?

Output: Neither positive nor negative.

8.4 Few-Shot Prompting

Few-shot prompting includes multiple examples within the prompt to demonstrate the desired behavior. This approach allows the model to infer patterns and generalize to new inputs. Few-shot prompting is widely used for classification, extraction, and reasoning tasks. Example includes,

Apple → Fruit; Carrot → Vegetable; Mango → Fruit; Potato → ?

Output: Vegetable.

8.5 Chain-of-Thought Prompting

Chain-of-Thought (CoT) prompting encourages the model to generate intermediate reasoning steps before arriving at a final answer. This technique has shown improvements in solving arithmetic, logical, and multi-step reasoning problems (Wei et al., 2022). Example includes,

Prompt: If a book costs ₹200 and a student receives a 25% discount, what is the final price? Explain your reasoning.

Output: 25% of ₹200 is ₹50. Final price = ₹200 – ₹50 = ₹150. Thus, the student pays ₹150.

8.6 Prompt Design Principles and Best Practices

Some commonly followed prompt engineering practices include:

- Use clear and specific instructions.
- Provide examples whenever necessary.
- Define the expected output format.
- Break complex tasks into smaller steps.
- Include relevant context to improve response quality.
- Experiment with different prompting techniques to obtain optimal results.

Carefully designed prompts can substantially enhance the effectiveness, reliability, and usability of Large Language Models in practical applications.

9 Challenges and Ethical Considerations

Despite their remarkable capabilities, Large Language Models (LLMs) face several technical, societal, and ethical challenges. Addressing these concerns is essential to ensure the safe, fair, and responsible deployment of AI systems in real-world applications.

9.1 Hallucinations in Large Language Models

Hallucinations occur when an LLM generates information that appears plausible and coherent but is factually incorrect, fabricated, or unsupported by reliable sources. Hallucinations may occur due to limited training data, a lack of domain-specific knowledge, or probabilistic text-generation mechanisms.

For example, a language model may generate non-existent references, incorrect dates, or fabricated research findings. Various approaches, such as Retrieval-Augmented Generation (RAG), human feedback, and fact-checking systems, have been proposed to reduce hallucinations and improve response reliability (Ji et al., 2023).

9.2 Bias and Fairness

LLMs learn patterns from large-scale datasets collected from diverse sources, which may contain social, cultural, or demographic biases. As a result, models may unintentionally produce outputs that reflect stereotypes or unfair treatment toward certain groups.

Biases in AI systems can affect decision-making across areas such as recruitment, healthcare, education, and financial services. Researchers continue to investigate methods for detecting, measuring, and mitigating bias to promote fairness and inclusiveness in AI applications (Bender et al., 2021).

9.3 Privacy and Security Concerns

Training datasets used for developing LLMs may contain sensitive or personally identifiable information. There is a potential risk that models could inadvertently reproduce memorized data during inference.

Additional security concerns include prompt injection attacks, data leakage, unauthorized access, and misuse of generated content. Implementing privacy-preserving techniques, access controls, and secure deployment practices is important for safeguarding user information and maintaining trust in AI systems.

9.4 Misinformation and Misuse

The ability of LLMs to generate highly realistic text can also be exploited for harmful purposes. Examples include the creation of misleading news articles, spam messages, phishing emails, and manipulated online content.

The rapid dissemination of AI-generated misinformation may negatively influence public opinion and decision-making. Therefore, mechanisms for content verification, source attribution, and responsible usage policies are increasingly being considered to minimize potential misuse.

9.5 Responsible and Ethical AI Practices

Responsible AI aims to ensure that intelligent systems are developed and deployed in ways that benefit society while minimizing potential harm. Ethical considerations for LLMs include transparency, accountability, fairness, privacy protection, and human oversight.

Organizations and researchers advocate establishing governance frameworks, ethical guidelines, and regular audits to monitor AI systems throughout their lifecycles. Incorporating human feedback, improving explainability, and adhering to regulatory standards can help build trustworthy, socially beneficial AI technologies.

10 APIs and LLM Integration

Large Language Models are often deployed as cloud-based services that can be accessed programmatically through Application Programming Interfaces (APIs). APIs enable developers to integrate language models into applications such as chatbots, document analysis systems, code assistants, and intelligent search engines. Understanding APIs and their use is, therefore, an important step in building practical AI-powered applications.

10.1 Introduction to APIs

An Application Programming Interface (API) is a set of rules and protocols that allows different software applications to communicate with each other. APIs provide a standardized mechanism for requesting services, exchanging information, and accessing the functionality of external systems.

For example, an application can send a prompt to an LLM API and receive a generated response without managing the underlying model architecture or computational resources. APIs simplify application development by allowing developers to utilize existing services efficiently. Examples of commonly used APIs include Weather APIs, Payment APIs, Mapping APIs, Large Language Model APIs.

10.2 REST APIs

Representational State Transfer (REST) is one of the most widely adopted architectural styles for designing web-based APIs. REST APIs facilitate communication between clients and servers using standard HTTP methods.

Common HTTP methods include:

Method	Purpose
GET	Retrieve data
POST	Send data
PUT	Update data
DELETE	Remove data

For example, a client application may send a POST request containing a user prompt to an LLM server and receive a generated response. REST APIs are popular because they are lightweight, scalable, and easy to integrate with a wide range of programming languages.

10.3 JSON Data Handling

JavaScript Object Notation (JSON) is a lightweight data-interchange format commonly used in API communication. JSON stores information as key-value pairs and is both human-readable and machine-readable.

Example JSON data:

```
{  
  "model": "gpt-4",  
  "prompt": "Explain transformers",  
  "temperature": 0.7  
}
```

Python provides built-in support for handling JSON data through the `json` module, enabling easy parsing and generation of JSON objects. JSON has become the standard format for exchanging information between client applications and modern AI services.

10.4 Making API Requests using Python

Python offers several libraries for interacting with APIs, among which the `requests` library is one of the most widely used.

Example:

```
import requests  
  
response = requests.get("https://api.example.com/data")  
  
print(response.status_code)
```

The response received from an API can then be processed and converted into Python dictionaries or other data structures for further analysis. Python's simplicity and extensive ecosystem make it a preferred language for integrating AI services into applications.

10.5 OpenAI APIs

OpenAI provides APIs that allow developers to access powerful language models for tasks such as text generation, summarization, code assistance, and conversational AI. Typical API functionalities include Chat Completion, Text Generation, Embedding Generation, Speech Processing, and Image Generation. Developers can send prompts to OpenAI models and receive responses through simple API calls, making it easier to incorporate advanced AI capabilities into software applications.

OpenAI APIs have been widely adopted in research, education, customer support, and productivity tools due to their flexibility and ease of integration.

```
from openai import OpenAI

client=OpenAI()

response=client.responses.create(
    model="gpt-4o-mini",
    input="What is AI?"
)

print(response.output_text)
```

What is AI?

Artificial Intelligence is a field...

10.6 Hugging Face APIs

Hugging Face is a leading platform that hosts numerous pre-trained machine learning and natural language processing models. It provides APIs and inference endpoints that allow users to access transformer-based models without extensive computational resources. The Hugging Face ecosystem includes Transformers Library, Datasets Library, Tokenizers, Inference API, and Model Hub. Users can experiment with models such as BERT, GPT-2, T5, and LLaMA through simple API requests or Python libraries. Hugging Face has become an important resource for researchers and developers, promoting open-source collaboration and facilitating rapid experimentation with state-of-the-art language models.

```
from transformers import pipeline

pipe = pipeline("sentiment-analysis")

result = pipe("I love AI")

print(result)
```

[transformers] No model was supplied, defaulted to distilbert/distilbert-base-uncased-finetur
Using a pipeline without specifying a model name and revision in production is not recommende

Loading weights: 100%  104/104 [00:00<00:00, 1928.16it/s]

```
[{'label': 'POSITIVE', 'score': 0.9998302459716797}]
```

11 Introduction to AI Applications

The rapid advancements in Artificial Intelligence have led to numerous practical applications that help users automate tasks, enhance productivity, and improve decision-making. Recent progress in Large Language Models has enabled the creation of intelligent systems capable of understanding natural language, generating human-like responses, and interacting with users in real-time. Several frameworks and tools have also emerged to simplify the deployment of AI-powered applications. This section briefly introduces chatbots, conversational AI, and popular Python frameworks used for building interactive AI interfaces.

11.1 Chatbots and Conversational AI

Chatbots are software applications designed to simulate human conversations through text or voice interactions. Early chatbots relied primarily on predefined rules and decision trees, limiting their ability to understand diverse user inputs. However, advancements in Natural Language Processing and Large Language Models have significantly improved conversational capabilities. Modern conversational AI systems leverage transformer-based architectures and large-scale language models to generate contextually relevant and coherent responses. These systems can understand user intent, maintain conversational context, answer questions, summarize information, and assist users in performing various tasks.

Conversational AI has found applications in numerous domains, including Customer support systems, Virtual assistants, Educational platforms, Healthcare services, Recommendation systems, and research assistance tools.

Examples of contemporary conversational AI systems include ChatGPT, Gemini, Claude, and Microsoft Copilot. Their ability to engage in natural dialogues has contributed to the increasing adoption of AI-powered assistants across industries.

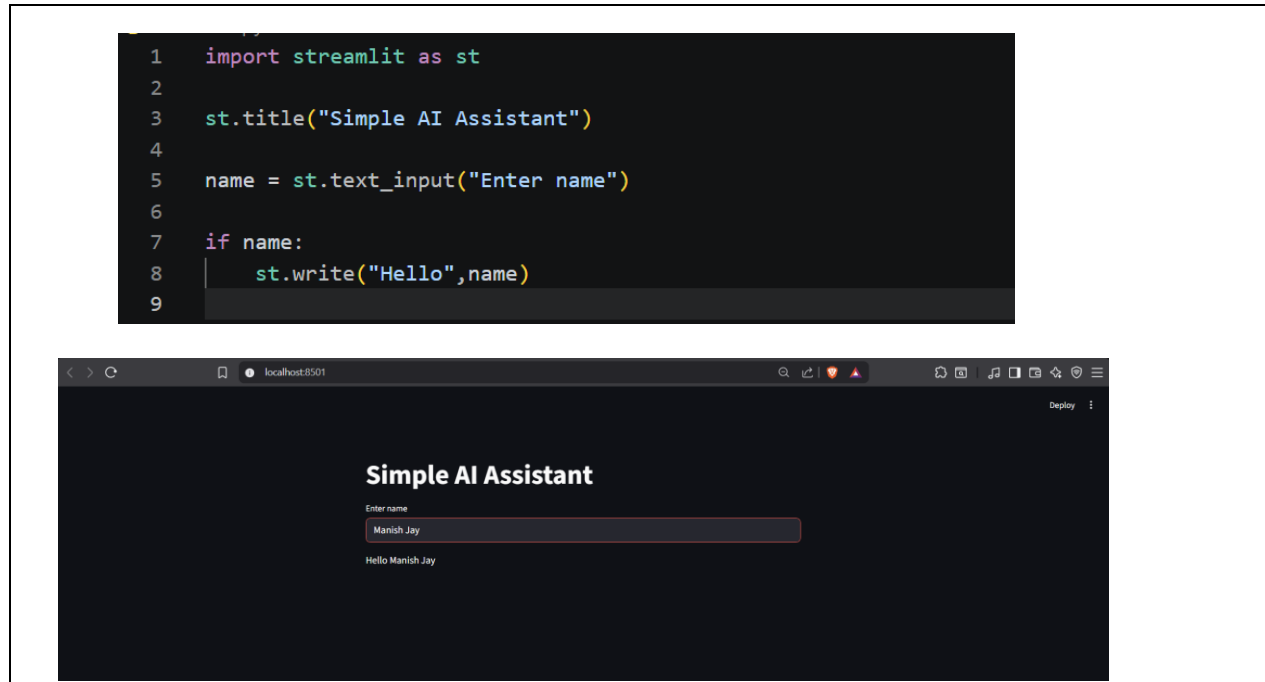
11.2 Streamlit Framework

Streamlit is an open-source Python framework designed for rapidly developing and deploying interactive web applications for data science and machine learning projects. It provides a simple, intuitive interface that enables developers to create web applications with only Python code, eliminating the need for extensive knowledge of front-end technologies such as HTML, CSS, or JavaScript.

Streamlit offers various built-in components, including Text input boxes, Buttons, Dropdown menus, Sliders, Data tables, Charts and visualizations, and file uploaders.

A simple Streamlit application can be developed using only a few lines of code.

Example:



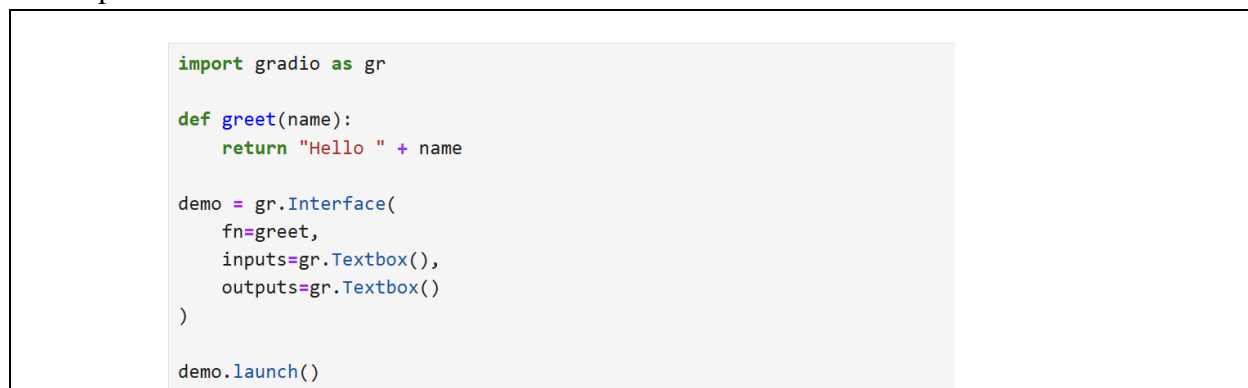
Streamlit is particularly useful for prototyping machine learning models, visualizing datasets, and developing AI-based dashboards. Its ease of deployment and integration with libraries such as Pandas, Matplotlib, Scikit-learn, and OpenAI APIs makes it a popular choice among researchers and developers.

11.3 Gradio Framework

Gradio is another open-source Python framework that simplifies the creation of user interfaces for machine learning and artificial intelligence models. It is specifically designed to provide quick demonstrations and interactive interfaces for AI applications.

Gradio supports a wide range of input and output components, including text inputs, images, audio files, videos, dataframes, and chat interfaces. One of Gradio's key advantages is its ability to create shareable web interfaces with minimal coding effort.

Example:



```
* Running on local URL: http://127.0.0.1:7860
* To create a public link, set `share=True` in `launch()`.
```

name	output	
Manish Jay	Hello Manish Jay	
Clear	Submit	Flag

Gradio has gained popularity in the machine learning community for its seamless integration with Hugging Face models and its ability to rapidly showcase AI prototypes.

11.4 Building Simple AI Applications

The availability of pre-trained language models, APIs, and lightweight web frameworks has made it easier to develop AI-powered applications without requiring extensive computational infrastructure.

Simple AI applications can be built by combining components such as Large Language Models, Python programming, API services, Streamlit or Gradio interfaces, and external knowledge sources. Examples of beginner-level AI applications include question-answering chatbots, document summarizers, resume analyzers, language translators, code assistants, and educational tutors.

The theoretical understanding gained during the initial phase of this project provides a foundation for developing practical AI applications in subsequent stages of the project. Future work may involve integrating language models with user interfaces, external datasets, and retrieval systems to create more robust and domain-specific intelligent applications.

12 Mid-Term Progress Summary

The initial phase of the project primarily focused on developing a strong theoretical foundation in Artificial Intelligence, Natural Language Processing, and Large Language Models. During this period, an extensive literature review was conducted to understand the evolution of language models, ranging from early rule-based systems to modern transformer-based architectures.

Fundamental Python programming concepts, including functions, loops, dictionaries, file handling, and scientific computing libraries such as NumPy, Pandas, and Matplotlib, were studied to strengthen the programming skills needed to implement AI applications. Concepts related to Artificial Intelligence, Machine Learning, Deep Learning, and Natural Language Processing were also explored to establish a comprehensive understanding of the technologies underlying Large Language Models.

Considerable effort was devoted to studying transformer architectures, including encoder-decoder mechanisms, self-attention, multi-head attention, and positional encoding techniques. Popular transformer-based models such as BERT and GPT were reviewed to understand their architectures, training methodologies, and practical applications.

Additionally, text generation techniques, prompt engineering strategies, ethical considerations, and challenges associated with Large Language Models were examined. Various API-based deployment approaches, including OpenAI APIs and Hugging Face APIs, were also investigated to gain familiarity with integrating pre-trained language models into real-world applications.

The initial exploration of frameworks such as Streamlit and Gradio was undertaken to understand their capabilities for developing interactive AI-powered applications. Based on the theoretical knowledge acquired during this phase, the subsequent stages of the project will focus on selecting an appropriate use case, designing the application architecture, integrating language model APIs, and developing a prototype system.

13 Conclusion

Large Language Models have emerged as one of the most significant advancements in Artificial Intelligence, demonstrating remarkable capabilities in understanding, processing, and generating human language. Their development has been driven by decades of research, beginning with rule-based systems and statistical language models, progressing through neural networks, attention mechanisms, and ultimately transformer architectures. Models such as BERT and GPT have further expanded the possibilities of language-based applications by enabling efficient learning from large-scale textual datasets and supporting a wide range of natural language processing tasks.

This report provided a comprehensive overview of the fundamental concepts underlying Large Language Models, including tokenization, embeddings, self-attention, transformer architectures, text generation techniques, prompt engineering, and ethical considerations. In addition, modern approaches to integrating language models into practical applications via APIs, the Streamlit and Gradio frameworks were discussed.

- The activities undertaken during the initial phase of the project primarily focused on acquiring theoretical knowledge and understanding the technologies required for developing AI-powered applications. This study has provided a strong foundation for the subsequent stages of the program, which will involve selecting an appropriate use case, integrating pre-trained language models, designing user interfaces, and implementing prototype applications. The knowledge gained from this phase is expected to facilitate the development of practical, efficient solutions that leverage Large Language Models in real-world scenarios.

14 References

- Abid, A., Abdalla, A., Abid, A., Khan, D., Alfozan, A., Zou, J., and Zou, J. (2019). *Gradio: Hassle-Free Sharing and Testing of Machine Learning Models in the Wild*.
- Bender, E. M., Gebru, T., McMillan-Major, A., and Shmitchell, S. (2021). *On the Dangers of Stochastic Parrots: Can Language Models Be Too Big?* Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency, pp. 610–623.
- Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., et al. (2020). *Language Models are Few-Shot Learners*. Advances in Neural Information Processing Systems, 33, 1877–1901.
- Devlin, J., Chang, M. W., Lee, K., and Toutanova, K. (2019). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. Proceedings of NAACL-HLT 2019, pp. 4171–4186.
- Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures*. Doctoral Dissertation, University of California, Irvine.
- Goodfellow, I., Bengio, Y., and Courville, A. (2016). *Deep Learning*. MIT Press, Cambridge, Massachusetts.
- Harris, C. R., Millman, K. J., van der Walt, S. J., et al. (2020). *Array Programming with NumPy*. Nature, 585(7825), 357–362.
- Hochreiter, S., and Schmidhuber, J. (1997). *Long Short-Term Memory*. Neural Computation, 9(8), 1735–1780.
- Hunter, J. D. (2007). *Matplotlib: A 2D Graphics Environment*. Computing in Science & Engineering, 9(3), 90–95.
- Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., et al. (2023). *Survey of Hallucination in Natural Language Generation*. ACM Computing Surveys, 55(12), 1–38.
- Jurafsky, D., and Martin, J. H. (2023). *Speech and Language Processing*. 3rd Edition (Draft). Stanford University.
- McCarthy, J., Minsky, M., Rochester, N., and Shannon, C. (1955). *A Proposal for the Dartmouth Summer Research Project on Artificial Intelligence*.
- McKinney, W. (2010). *Data Structures for Statistical Computing in Python*. Proceedings of the 9th Python in Science Conference, 51–56.
- Mikolov, T., Chen, K., Corrado, G., and Dean, J. (2013). *Efficient Estimation of Word Representations in Vector Space*. arXiv:1301.3781.



- Mitchell, T. M. (1997). *Machine Learning*. McGraw-Hill Education.
- OpenAI. (2024). *OpenAI API Documentation*.
- Pennington, J., Socher, R., and Manning, C. D. (2014). *GloVe: Global Vectors for Word Representation*. Proceedings of EMNLP 2014.
- Radford, A., Narasimhan, K., Salimans, T., and Sutskever, I. (2018). *Improving Language Understanding by Generative Pre-Training*. OpenAI Technical Report.
- Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., and Sutskever, I. (2019). *Language Models are Unsupervised Multitask Learners*. OpenAI Technical Report.
- Richardson, M., and Streamlit Contributors. (2024). *Streamlit Documentation*.
- Russell, S., and Norvig, P. (2021). *Artificial Intelligence: A Modern Approach*. 4th Edition. Pearson Education.
- Van Rossum, G., and Drake, F. L. (2009). *Python 3 Reference Manual*. CreateSpace Independent Publishing Platform.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. (2017). *Attention Is All You Need*. Advances in Neural Information Processing Systems, 30.
- Wei, J., Wang, X., Schuurmans, D., et al. (2022). *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*. Advances in Neural Information Processing Systems, 35.
- Wolf, T., Debut, L., Sanh, V., et al. (2020). *Transformers: State-of-the-Art Natural Language Processing*. Proceedings of EMNLP 2020: System Demonstrations, 38–45.

